SUBMITTED BY: NIKHIL NARAYAN NAIK (BSc.CS 4TH Sem)

REGD NO: GGV/24/05141

ROLL NO: 24070141

1.Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

```
import java.io.*;
import java.util.*;

class Farmer {
    String name;
    String fruit;
    String city;
    int quantity;

    Farmer(String name, String fruit, String city, int quantity) {
        this.name = name;
        this.fruit = fruit;
        this.city = city;
        this.quantity = quantity;
    }
}

public class practical1{

    public static void writeToFile(List<Farmer> list, String filename) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(filename))) {
            writer.write(String.format("%-15s %-15s %-15s %-10s\n", "Name",
"Fruit", "City", "Quantity"));
            writer.write("-------------------------------------------------
---------\n");

            for (Farmer f : list) {
                writer.write(String.format("%-15s %-15s %-15s %-10d\n",
                        f.name, f.fruit, f.city, f.quantity));
            }
        } catch (IOException e) {
            System.out.println("Error writing file: " + e.getMessage());
        }
    }

    public static void search(List<Farmer> list, String searchName) {
```

```java
        boolean found = false;

        for (Farmer f : list) {
            if (f.name.equalsIgnoreCase(searchName)) {
                System.out.println("Farmer Found:");
                System.out.printf("%-15s %-15s %-15s %-10s\n", "Name",
"Fruit", "City", "Quantity");
                System.out.println("----------------------------------------
-------------------");
                System.out.printf("%-15s %-15s %-15s %-10d\n",
                        f.name, f.fruit, f.city, f.quantity);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Farmer not found.");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Farmer> farmers = new ArrayList<>();

        System.out.println("Enter details for 10 farmers:");
        for (int i = 0; i < 10; i++) {
            System.out.println("\nFarmer " + (i + 1));

            System.out.print("Name: ");
            String name = sc.nextLine();

            System.out.print("Fruit: ");
            String fruit = sc.nextLine();

            System.out.print("City: ");
            String city = sc.nextLine();

            System.out.print("Quantity: ");
            int quantity = sc.nextInt();
            sc.nextLine();


            farmers.add(new Farmer(name, fruit, city, quantity));
        }


        writeToFile(farmers, "before_sort.txt");
```

```java
        farmers.sort(Comparator.comparingInt(f -> f.quantity));
        writeToFile(farmers, "after_sort.txt");

        System.out.print("\nEnter farmer name to search: ");
        String searchName = sc.nextLine();
        search(farmers, searchName);

        sc.close();
    }
}
```

OUTPUT:

```
Enter details for 10 farmers:

Farmer 1
Name: Ram
Fruit: apple
City: Raipur
Quantity: 50

Farmer 2
Name: Ayush
Fruit: watermelon
City: Ranchi
Quantity: 70

Farmer 3
Name: Mahesh
Fruit: mango
City: Mumbai
Quantity: 90

Farmer 4
Name: Golu
Fruit: guava
City: Surat
Quantity: 65

Farmer 5
Name: Vishal
Fruit: apple
City: Kanpur
Quantity: 75
```

```
Farmer 6
Name: Sikhar
Fruit: Guava
City: Raipur
Quantity: 65

Farmer 7
Name: Nilesh
Fruit: watermelon
City: Delhi
Quantity: 55

Farmer 8
Name: Hari
Fruit: mango
City: Delhi
Quantity: 80

Farmer 9
Name: Yash
Fruit: orange
City: Manali
Quantity: 70

Farmer 10
Name: Dipak
Fruit: lichi
City: Mumbai
Quantity: 85

Enter farmer name to search: Hari
Farmer Found:
Name            Fruit           City            Quantity
-------------------------------------------------------
Hari            mango           Delhi           80
```

before_sort.txt

```
Name            Fruit           City            Quantity
-------------------------------------------------------
Ram             apple           Raipur          50
Ayush           watermelon      Ranchi          70
Mahesh          mango           Mumbai          90
Golu            guava           Surat           65
Vishal          apple           Kanpur          75
Sikhar          Guava           Raipur          65
Nilesh          watermelon      Delhi           55
Hari            mango           Delhi           80
Yash            orange          Manali          70
Dipak           lichi           Mumbai          85
```

after_sort.txt

```
Name            Fruit           City            Quantity
-------------------------------------------------------------
Ram             apple           Raipur          50
Nilesh          watermelon      Delhi           55
Golu            guava           Surat           65
Sikhar          Guava           Raipur          65
Ayush           watermelon      Ranchi          70
Yash            orange          Manali          70
Vishal          apple           Kanpur          75
Hari            mango           Delhi           80
Dipak           lichi           Mumbai          85
Mahesh          mango           Mumbai          90
```

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

```java
import java.util.Scanner;

class Fruit {
    String name;
    int quantity;
    String quality;

    Fruit(String name, int quantity, String quality) {
        this.name = name;
        if (quantity > 1000) {
            this.quantity = quantity / 2;
        } else {
            this.quantity = quantity;
        }

        this.quality = quality.toLowerCase();
    }
    void display() {
        System.out.println(name + "\t" + quantity + "\t" + quality);
    }
}
```

```java
public class practical2{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Fruit[] fruits = new Fruit[3];
        String[] fruitNames = {"Mango", "Apple", "Papaya"};

        for (int i = 0; i < 3; i++) {
            System.out.print("Enter quantity for " + fruitNames[i] + ": ");
            int qty = sc.nextInt();

            System.out.print("Enter quality (good/bad/average): ");
            String quality = sc.next();

            fruits[i] = new Fruit(fruitNames[i], qty, quality);
        }

        System.out.println("\nStored Data:");
        System.out.println("Fruit\tQuantity\tQuality");
        for (Fruit f : fruits) {
            f.display();
        }

        System.out.print("Enter fruit name to search (Mango,Apple,Papaya):");
        String searchName = sc.next();

        System.out.print("Enter quality to search (good/bad/average):");
        String searchQuality = sc.next().toLowerCase();

        boolean found = false;
        for (Fruit f : fruits) {
            if (f.name.equalsIgnoreCase(searchName) &&
                f.quality.equals(searchQuality)) {
                System.out.println("\nMatch Found:");
                f.display();
                found = true;
            }
        }
        if (!found) {
            System.out.println("\nNo matching fruit found.");
        }
        sc.close();
    }
}
```

Output

```
Enter quantity for Mango: 50
Enter quality (good/bad/average): good
Enter quantity for Apple: 70
Enter quality (good/bad/average): good
Enter quantity for Papaya: 55
Enter quality (good/bad/average): bad

Stored Data:
Fruit   Quantity        Quality
Mango   50      good
Apple   70      good
Papaya  55      bad
Enter fruit name to search (Mango,Apple,Papaya):mango
Enter quality to search (good/bad/average):good

Match Found:
Mango   50      good
```

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively. The available orchards in a tree-view. Search for a given fruit in the available orchards after taking input from the user & print the tree.

```java
import java.util.*;

class OrchardNode {
    String orchardName;
    String fruit;
    String quality;
    List<OrchardNode> children;

    OrchardNode(String orchardName, String fruit, String quality) {
        this.orchardName = orchardName;
        this.fruit = fruit;
        this.quality = quality;
        this.children = new ArrayList<>();
    }
}

public class practical3{
```

```java
    static void printTree(OrchardNode root, String searchFruit) {

        if (root.orchardName.equals("Orchards")) {
            System.out.println(root.orchardName);
        } else {

            if (root.fruit.equalsIgnoreCase(searchFruit)) {
                System.out.println("├─  "+ root.orchardName +
                    " (" + root.fruit + ", " + root.quality + ") [FOUND]");
            } else {
                System.out.println("├─  "+ root.orchardName +
                    " (" + root.fruit + ", " + root.quality + ")");
            }
        }

        for (OrchardNode child : root.children) {
            printTree(child, searchFruit);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        OrchardNode root = new OrchardNode("Orchards", "", "");

        root.children.add(new OrchardNode("A", "Apple", "good"));
        root.children.add(new OrchardNode("B", "Litchi", "good"));
        root.children.add(new OrchardNode("C", "Mango", "good"));
        root.children.add(new OrchardNode("D", "Apple", "bad"));

        System.out.print("Enter fruit to search: ");
        String searchFruit = sc.nextLine();

        System.out.println("\nTree View:");
        printTree(root, searchFruit);

        sc.close();
    }
}
```

output

```
Enter fruit to search: apple

Tree View:
Orchards
├─  A (Apple, good) [FOUND]
├─  B (Litchi, good)
├─  C (Mango, good)
├─  D (Apple, bad) [FOUND]
```

4. Take a map provided as a graph & display all the subtrees of connected points.

```java
import java.util.*;

public class practical4 {

    // DFS to find one connected component
    static void dfs(int node, boolean[] visited, List<List<Integer>> graph,
List<Integer> component) {
        visited[node] = true;
        component.add(node);

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited, graph, component);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int n = sc.nextInt();
        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        // Adjacency list
        List<List<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        // Input edges
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u); // Undirected graph
        }

        boolean[] visited = new boolean[n];

        System.out.println("\nSubtrees (Connected Components):");

        // Find all components
```

```java
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                List<Integer> component = new ArrayList<>();
                dfs(i, visited, graph, component);

                // Print one subtree
                System.out.println(component);
            }
        }

        sc.close();
    }
}
```

output

```
Enter number of vertices: 4
Enter number of edges: 3
Enter edges (u v):
0 1
1 2
2 3
```

5. Enter entry point & goal point & produce the matching subtree of the graph.

```java
import java.util.*;

public class practical5{

    // DFS to find path
    static boolean dfs(int current, int goal, boolean[] visited,
                       List<List<Integer>> graph, List<Integer> path) {

        visited[current] = true;
        path.add(current);

        // If goal found
        if (current == goal) {
            return true;
        }

        for (int neighbor : graph.get(current)) {
            if (!visited[neighbor]) {
```

```java
                if (dfs(neighbor, goal, visited, graph, path)) {
                    return true;
                }
            }
        }

        // Backtrack if not found
        path.remove(path.size() - 1);
        return false;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u); // undirected
        }

        System.out.print("Enter entry point: ");
        int start = sc.nextInt();

        System.out.print("Enter goal point: ");
        int goal = sc.nextInt();

        boolean[] visited = new boolean[n];
        List<Integer> path = new ArrayList<>();

        if (dfs(start, goal, visited, graph, path)) {
            System.out.println("\nMatching Subtree (Path):");

            // Print as tree/path
```

```java
            for (int node : path) {
                System.out.print(node + " -> ");
            }
            System.out.println("Goal");
        } else {
            System.out.println("\nNo path exists between given points.");
        }

        sc.close();
    }
}
```

output

```
Enter number of vertices: 5
Enter number of edges: 4
Enter edges (u v):
0 1
1 2
2 3
3 4
Enter entry point: 2
Enter goal point: 4

Matching Subtree (Path):
2 -> 3 -> 4 -> Goal
```

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```java
import java.util.*;

class Edge {
    int target, weight;

    Edge(int target, int weight) {
        this.target = target;
        this.weight = weight;
    }
}
```

```java
public class practical6{

    static void dijkstra(List<List<Edge>> graph, int start, int end) {
        int n = graph.size();

        int[] dist = new int[n];
        int[] parent = new int[n];

        Arrays.fill(dist, Integer.MAX_VALUE);
        Arrays.fill(parent, -1);

        dist[start] = 0;

        PriorityQueue<int[]> pq = new
PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
        pq.add(new int[]{start, 0});

        while (!pq.isEmpty()) {
            int[] current = pq.poll();
            int node = current[0];
            int d = current[1];

            if (d > dist[node]) continue;

            for (Edge edge : graph.get(node)) {
                int newDist = dist[node] + edge.weight;

                if (newDist < dist[edge.target]) {
                    dist[edge.target] = newDist;
                    parent[edge.target] = node;
                    pq.add(new int[]{edge.target, newDist});
                }
            }
        }

        // If unreachable
        if (dist[end] == Integer.MAX_VALUE) {
            System.out.println("No path exists.");
            return;
        }

        // Reconstruct path
        List<Integer> path = new ArrayList<>();
        for (int v = end; v != -1; v = parent[v]) {
            path.add(v);
        }
        Collections.reverse(path);
```

```java
        // Output
        System.out.println("\nShortest Path:");
        for (int node : path) {
            System.out.print(node + " -> ");
        }
        System.out.println("End");

        System.out.println("Total Cost: " + dist[end]);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        List<List<Edge>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        System.out.println("Enter edges (u v weight):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            int w = sc.nextInt();

            graph.get(u).add(new Edge(v, w));
            graph.get(v).add(new Edge(u, w)); // undirected
        }

        System.out.print("Enter start node: ");
        int start = sc.nextInt();

        System.out.print("Enter end node: ");
        int end = sc.nextInt();

        dijkstra(graph, start, end);

        sc.close();
    }
}
```

Output

```
Enter number of vertices: 5
Enter number of edges: 6
Enter edges (u v weight):
0 1 4
0 2 1
2 1 2
1 3 1
2 3 5
3 4 3
Enter start node: 0
Enter end node: 4

Shortest Path:
0 -> 2 -> 1 -> 3 -> 4 -> End
Total Cost: 7
```

7.Write a program to demonstrate the use of all keywords made for exception handling.

```java
import java.io.*;

class practical7 {

    // Method using 'throws'
    static void checkAge(int age) throws ArithmeticException {
        if (age < 18) {
            // using 'throw'
            throw new ArithmeticException("Not eligible to vote");
        } else {
            System.out.println("Eligible to vote");
        }
    }

    public static void main(String[] args) {

        try {
            // Example 1: Arithmetic Exception
            int a = 10, b = 0;
            int result = a / b; // will cause exception
            System.out.println("Result: " + result);
```

```
        }
        // multiple catch blocks
        catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " +
e.getMessage());
        }
        catch (Exception e) {
            System.out.println("Caught General Exception");
        }
        finally {
            // always executes
            System.out.println("Finally block executed");
        }

        // Example 2: Using throw and throws
        try {
            checkAge(15);
        } catch (ArithmeticException e) {
            System.out.println("Exception from method: " + e.getMessage());
        }

        // Example 3: File handling exception
        try {
            BufferedReader br = new BufferedReader(new
FileReader("test.txt"));
            System.out.println(br.readLine());
            br.close();
        } catch (IOException e) {
            System.out.println("File not found or error reading file");
        }

        System.out.println("Program continues...");
    }
}
```

Output

```
Caught ArithmeticException: / by zero
Finally block executed
Exception from method: Not eligible to vote
File not found or error reading file
Program continues...
```

8. Write a program to withdraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

```java
import java.util.Scanner;

class NoMoneyException extends Exception {
    public NoMoneyException(String message) {
        super(message);
    }
}

class BankAccount {
    double balance;

    BankAccount(double balance) {
        this.balance = balance;
    }

    void withdraw(double amount) throws NoMoneyException {
        if (amount > balance) {
            throw new NoMoneyException("Insufficient balance!");
        } else {
            balance -= amount;
            System.out.println("Withdrawal successful!");
            System.out.println("Remaining Balance: " + balance);
        }
    }
}

public class practical8{
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter initial balance: ");
        double balance = sc.nextDouble();

        BankAccount acc = new BankAccount(balance);

        System.out.print("Enter amount to withdraw: ");
        double amount = sc.nextDouble();

        try {
            acc.withdraw(amount);
        }
        catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        }
        finally {
```

```
            System.out.println("Transaction completed.");
        }

        sc.close();
    }
}
```

Output

```
Enter initial balance: 2000
Enter amount to withdraw: 2500
Exception: Insufficient balance!
Transaction completed.
```

9. Write a program which enables to use a platform by a train one at a time.

```
class Platform {

    // synchronized method ensures only one thread at a time
    synchronized void usePlatform(String trainName) {
        System.out.println(trainName + " is entering the platform...");

        try {
            // simulate time taken by train at platform
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println(trainName + " is leaving the platform...\n");
    }
}

// Train class extending Thread
class Train extends Thread {
    Platform platform;
    String trainName;

    Train(Platform platform, String trainName) {
        this.platform = platform;
        this.trainName = trainName;
    }

    public void run() {
        platform.usePlatform(trainName);
    }
}
```

```java
// Main class
public class practical9 {
    public static void main(String[] args) {

        Platform platform = new Platform();

        // Creating multiple trains (threads)
        Train t1 = new Train(platform, "Train A");
        Train t2 = new Train(platform, "Train B");
        Train t3 = new Train(platform, "Train C");

        // Starting threads
        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output

```
Train A is entering the platform...
Train A is leaving the platform...

Train B is entering the platform...
Train B is leaving the platform...

Train C is entering the platform...
Train C is leaving the platform...
```

10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

```java
class Restaurant {
    private boolean orderPlaced = false;

    // Waiter places order
    synchronized void placeOrder(String order) {
        while (orderPlaced) {
            try {
                wait(); // wait if kitchen is busy
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }

        System.out.println("Waiter placed order: " + order);
```

```java
        orderPlaced = true;

        notify(); // notify chef
    }

    // Chef prepares order
    synchronized void prepareOrder() {
        while (!orderPlaced) {
            try {
                wait(); // wait if no order
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }

        System.out.println("Chef is preparing and delivering order...");
        orderPlaced = false;

        notify(); // notify waiter
    }
}

// Waiter Thread
class Waiter extends Thread {
    Restaurant restaurant;

    Waiter(Restaurant r) {
        restaurant = r;
    }

    public void run() {
        String orders[] = {"Pizza", "Burger", "Pasta"};

        for (String order : orders) {
            restaurant.placeOrder(order);

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

// Chef Thread
class Chef extends Thread {
    Restaurant restaurant;
```

```java
    Chef(Restaurant r) {
        restaurant = r;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            restaurant.prepareOrder();

            try {
                Thread.sleep(1500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

// Main class
public class practical10{
    public static void main(String[] args) {

        Restaurant r = new Restaurant();

        Waiter waiter = new Waiter(r);
        Chef chef = new Chef(r);

        waiter.start();
        chef.start();
    }
}
```

Output

```
Waiter placed order: Pizza
Chef is preparing and delivering order...
Waiter placed order: Burger
Chef is preparing and delivering order...
Waiter placed order: Pasta
Chef is preparing and delivering order...
```